

bomblab

000000

0002024201593

00	phase_1	phase_2	phase_3	phase_4	phase_5	phase_6	secret_phase
7	1	1	1	1	1	1	1

scoreboard

2024201593	0	2	0	0	0	1	0	0	0	0	0	0	0	0	3	7
------------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

0000

phase_1

If abstraction is the definition of beauty, are those of us chasing after clarity a representation of ugly?

00000000call00strings_not_equal00000000000000000000x/s \$rsi00000000000000

phase_2

349703 719796 252009 471328

python000000

```
def phase_2(input_str):
    #scanf004000000147d-148900
    try:
        inputs = list(map(int, input_str.split()))
    except ValueError:
        explode_bomb()

    #00000000004:cmp $0x4,%eax 00
    if len(inputs) != 4:
        explode_bomb()

    #matA0matB
    matA = []
    matB = []

    #00000000
    result = []
```

```

for row in matA:
    sum_val = 0
    for i in range(3):
        sum_val += row[i] * matB[i][0]
    result.append(sum_val)

#
for input_val, target_val in zip(inputs, result):
    if input_val != target_val:
        explode_bomb()

print(1)

```

AB

phase_3

5 -429

python

```

def phase_3(input_str):
    try:
        parts = input_str.strip().split()
        if len(parts) != 3:
            explode_bomb()
        idx = int(parts[0])
        char = parts[1]
        val = int(parts[2])
        #
        if len(char) != 1:
            explode_bomb()
    except (ValueError, IndexError):
        explode_bomb()

    # idx
    if idx > 7:
        explode_bomb()

    #
    answer_char = None
    answer_val = None
    mask = 0x20

    if idx == 0:
        answer_char = 0x76
        answer_val = 0xfc
    elif idx == 1:
        answer_char = 0x76

```

```
p/x $eax mask cmp 0x70000000 0x70000000 input_val2 0x0
0xfc 252 15b5: je 169c
```

31 CA

```
def func4_1(n):
    if n <= 0:
        return 0
    if n == 1:
        return 1
    return 2 * func4_1(n - 1) + 1

def func4_2(n, target, start, end, aux, buffer):
    if n == 1:
        # n=1 start end
        buffer.append(chr(start))
        buffer.append(chr(end))
        return
    # limit = func4_1(n-1)
    limit = func4_1(n - 1)
    if target <= limit:
        #(start, aux, end)
        func4_2(n - 1, target, start, aux, end, buffer)
    elif target == limit + 1:
        #start\end
        buffer.append(chr(start))
        buffer.append(chr(end))
        return
    else:
        #(aux, end, start)
        new target = target - limit - 1
```

```
func4_2(n - 1, new_target, aux, end, start, buffer)

def phase_4(input_str):
    try:
        parts = input_str.strip().split()
        if len(parts) != 2:
            explode_bomb()
        input_int = int(parts[0])
        input_str = parts[1]
    except (ValueError, IndexError):
        explode_bomb()

    target_int = func4_1(5)
    if input_int != target_int:
        explode_bomb()

    #000000002
    if len(input_str) != 2:
        explode_bomb()

    #
    buffer = []
    # edi=5, esi=0x16(22), edx=0x41('A'), ecx=0x43('C'), r8d=0x42('B')
    r9=buffer
    func4_2(n=5, target=22, start=65, end=67, aux=66, buffer=buffer)
    target_str = ''.join(buffer)

    if input_str != target_str:
        explode_bomb()

    print(1)
```

ABC

phase_5

>74598

```
def phase_5(input_str):
    if len(input_str) != 6:
        explode_bomb()

    target_array = []

    target_str =
    result = []
    for char in input_str:
        char_ascii = ord(char)
        index = (char_ascii + 0xf) & 0xf
```

000000060000000000000000ASCII0xf 0xf 00000000000000000000000000000000
00000000000000

6 1 3 2 4 5 defuse

```
(gdb) set $node_head = 0x55555555A210
(gdb) set $cur = $node_head
(gdb) set $n = 1
(gdb) while $n <= 6
>printf "[%d] | %x=%lx | %s(val)=%d\n", $n, $cur, *(int*)$cur
>set $next = *(long*)($cur + 8)
>if $next == 0
>break
>end
>set $cur = $next
>set $n = $n + 1
>end

[[1] | %x=0x55555555a210 | %s(val)=777
[[2] | %x=0x55555555a220 | %s(val)=73
[[3] | %x=0x55555555a230 | %s(val)=371
[[4] | %x=0x55555555a240 | %s(val)=570
[[5] | %x=0x55555555a250 | %s(val)=531
[[6] | %x=0x55555555a160 | %s(val)=610
```

33022

5 / 8

```

0x0000000000000100 0x0000000000000000 00000000 [0,0,1,0,0,1,0,0], # row0 (0) [0,0,0,1,0,0,0,1], #
row1 (1) [1,0,1,0,0,1,0,0], # row2 (2) [1,0,0,0,0,0,0,0], # row3 (3) [0,1,0,0,1,0,1,0], # row4 (4)
[1,0,0,1,1,0,0,0], # row5 (5) [0,0,0,0,0,1,0,1], # row6 (6) [0,1,0,0,0,0,0,0], # row7 (7) 0000 (gdb) print
(int[8])($rsp) $5 = {-2, -1, 1, 2, 2, 1, -1, -2} (gdb) print (int[8])($rsp+0x20) $6 = {1, 2, 2, 1, -1, -2, -2, -1} (gdb) print
(int[8])($rsp+0x40) $7 = {-1, 0, 0, 1, 1, 0, 0, -1} (gdb) print (int[8])($rsp+0x60) $8 = {0, 1, 1, 0, 0, -1, -1, 0}000000
00000000000000000000000000000000

```

1. 00000003000idx
2. 000000 = (esi+C[idx], edx+D[idx])
3. 00000000000000010000
4. 000000 = (esi+A[idx], edx+B[idx])
5. 00000000000010000 00000000000100000000
6. 00000000000020000004070

0000000python00

```

from collections import deque

# 000001=000=0000
map_grid = [
    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1], # 0
    [1, 0, 0, 1, 0, 0, 1, 0, 0, 1], # 1
    [1, 0, 0, 0, 1, 0, 0, 0, 1, 1], # 2
    [1, 1, 0, 1, 0, 0, 1, 0, 0, 1], # 3
    [1, 1, 0, 0, 0, 0, 0, 0, 0, 1], # 4
    [1, 0, 1, 0, 0, 1, 0, 1, 0, 1], # 5000005008
    [1, 1, 0, 0, 1, 1, 0, 0, 0, 1], # 6
    [1, 0, 0, 0, 0, 0, 1, 0, 1, 1], # 7
    [1, 0, 1, 0, 0, 0, 0, 0, 0, 1], # 8
    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1] # 9
]

A = [-2, -1, 1, 2, 2, 1, -1, -2]
B = [1, 2, 2, 1, -1, -2, -2, -1]
C = [-1, 0, 0, 1, 1, 0, 0, -1]
D = [0, 1, 1, 0, 0, -1, -1, 0]

# 00000
DEST_ESI = 5
DEST_EDX = 8

# 00000000<20
MAX_LENGTH = 19

def find_valid_number_string():
    """
    00000000000(1,1)000000000(5,8)000<20
    """
    # 000BFS000(00esi, 00edx, 000)

```

```

queue = deque()
queue.append((1, 1, ""))

# 初始化
visited = set()
visited.add((1, 1))

while queue:
    current_esi, current_edx, num_str = queue.popleft()

    # 边界检查
    if len(num_str) >= MAX_LENGTH:
        continue

    # 枚举0-7和A/B/C/D
    for i in range(8):
        # 计算下一个位置
        temp_esi = current_esi + C[i]
        temp_edx = current_edx + D[i]
        # 检查是否在0-10范围内
        # if not (0 <= temp_esi < 10 and 0 <= temp_edx < 10):
        #     continue # 越界
        if map_grid[temp_esi][temp_edx] == 1:
            continue # 已访问

        # 计算下一个字符串
        new_esi = current_esi + A[i]
        new_edx = current_edx + B[i]
        # 检查是否在0-10范围内
        # if not (0 <= new_esi < 10 and 0 <= new_edx < 10):
        #     continue
        if map_grid[new_esi][new_edx] == 1:
            continue

        # 生成新字符串
        new_num_str = num_str + str(i)

        # 检查是否到达目标
        if new_esi == DEST_ESI and new_edx == DEST_EDX:
            return new_num_str

    # 加入队列
    if (new_esi, new_edx) not in visited:
        visited.add((new_esi, new_edx))
        queue.append((new_esi, new_edx, new_num_str))

# 返回结果
return "00000<2000000"

# 主函数
if __name__ == "__main__":
    result = find_valid_number_string()
    print(f"00000000{result}")

```

[illegible]

□□/□□/□□/□□

```

06phase8secret_phase
secret phase0049
horse gallops.txt
.....
1break
bomblab

```

□ □ □ □ □ □ □

lhy bomb lab report